Date: 30-03-2026

Datatypes:

Strings:

Backticks are “extended functionality” quotes. They allow us to embed variables and expressions into a string by wrapping them in ${…}, for example:

```
let name = "John";

// embed a variable

alert( `Hello, ${name}!` ); // Hello, John!

// embed an expression

alert( `the result is ${1 + 2}` ); // the result is 3
```

Please note that this can only be done in backticks. Other quotes don’t have this embedding functionality!

```
alert( "the result is ${1 + 2}" ); // the result is ${1 + 2} (double quotes do nothing)
```

Boolean (logical type):

The boolean type has only two values: true and false.

This type is commonly used to store yes/no values: true means “yes, correct”, and false means “no, incorrect”.

Boolean values also come as a result of comparisons:

```
let isGreater = 4 > 1;

alert( isGreater ); // true (the comparison result is "yes")
```

The null value:

The special null value does not belong to any of the types described above.

It forms a separate type of its own which contains only the null value:

```
let age = null;
```

In JavaScript, null is not a “reference to a non-existing object” or a “null pointer” like in some other languages.

It’s just a special value which represents “nothing”, “empty” or “value unknown”.

The code above states that age is unknown.

The undefined value:

The special value undefined also stands apart. It makes a type of its own, just like null.

The meaning of undefined is “value is not assigned”.

If a variable is declared, but not assigned, then its value is undefined:

```
let age;
alert(age); // shows "undefined"
```

Technically, it is possible to explicitly assign undefined to a variable.

Objects and Symbols:

The object type is special.

All other types are called “primitive” because their values can contain only a single thing (be it a string or a number or whatever). In contrast, objects are used to store collections of data and more complex entities.

The symbol type is used to create unique identifiers for objects. We have to mention it here for the sake of completeness, but also postpone the details till we know objects.

The typeof operator:

The typeof operator returns the type of the operand. It’s useful when we want to process values of different types differently or just want to do a quick check.

A call to typeof x returns a string with the type name

```
typeof undefined // "undefined"
typeof 0 // "number"
typeof 10n // "bigint"
typeof true // "boolean"
typeof "foo" // "string"
typeof Symbol("id") // "symbol"
typeof Math // "object"  (1)
typeof null // "object"  (2)
typeof alert // "function"  (3)
```

1. Math is a built-in object that provides mathematical operations.
2. The result of typeof null is "object". That’s an officially recognized error in typeof, coming from very early days of JavaScript and kept for compatibility. Definitely, null is not an object. It is a special value with a separate type of its own. The behavior of typeof is wrong here.
3. The result of typeof alert is "function", because alert is a function. We’ll study functions in the next chapters where we’ll also see that there’s no special “function” type in JavaScript. Functions belong to the object type. But typeof treats them differently, returning "function". That also comes from the early days of JavaScript. Technically, such behavior isn’t correct, but can be convenient in practice.

The typeof() syntax:

You may also come across another syntax: typeof(x). It’s the same as typeof x.

typeof is an operator, not a function. The parentheses here aren't a part of typeof. It's the kind of parentheses used for mathematical grouping.

Usually, such parentheses contain a mathematical expression, such as (2 + 2), but here they contain only one argument (x). Syntactically, they allow to avoid a space between the typeof operator and its argument, and some people like it.

Some people prefer typeof(x), although the typeof x syntax is much more common.

Interaction: alert, prompt, confirm:

alert:

It shows a message and waits for the user to press "OK".

The mini-window with the message is called a *modal window*. The word "modal" means that the visitor can't interact with the rest of the page, press other buttons, etc, until they have dealt with the window. In this case – until they press "OK".

prompt:

The function prompt accepts two arguments:

```
result = prompt(title, [default]);
```

It shows a modal window with a text message, an input field for the visitor, and the buttons OK/Cancel.

title

The text to show the visitor.

default

An optional second parameter, the initial value for the input field.

The square brackets in syntax [...]

The square brackets around default in the syntax above denote that the parameter is optional, not required.

The second parameter is optional, but if we don't supply it, Internet Explorer will insert the text "undefined" into the prompt.

Run this code in Internet Explorer to see:

```
let test = prompt("Test");
```

So, for prompts to look good in IE, we recommend always providing the second argument:

```
let test = prompt("Test", ''); // <-- for IE
```

confirm:

The syntax:

```
result = confirm(question);
```

The function confirm shows a modal window with a question and two buttons: OK and Cancel.

The result is true if OK is pressed and false otherwise.

All these methods are modal: they pause script execution and don't allow the visitor to interact with the rest of the page until the window has been dismissed.

There are two limitations shared by all the methods above:

1. The exact location of the modal window is determined by the browser. Usually, it's in the center.
2. The exact look of the window also depends on the browser. We can't modify it.

Type Conversions:

Most of the time, operators and functions automatically convert the values given to them to the right type.

For example, alert automatically converts any value to a string to show it. Mathematical operations convert values to numbers.

There are also cases when we need to explicitly convert a value to the expected type.

String Conversion:

String conversion happens when we need the string form of a value.

For example, alert(value) does it to show the value.

We can also call the String(value) function to convert a value to a string:

```
let value = true;

alert(typeof value); // boolean

value = String(value); // now value is a string "true"

alert(typeof value); // string
```

Numeric Conversion:

Numeric conversion in mathematical functions and expressions happens automatically.

For example, when division / is applied to non-numbers:

```
alert( "6" / "2" ); // 3, strings are converted to numbers
```

We can use the Number(value) function to explicitly convert a value to a number:

```
let str = "123";

alert(typeof str); // string

let num = Number(str); // becomes a number 123

alert(typeof num); // number
```

Numeric conversion rules:

Value	Becomes…
undefined	NaN
null	0
true and false	1 and 0
string	Whitespaces (includes spaces, tabs \t, newlines \n etc.) from the start and end are removed. If the remaining string is empty, the result is 0. Otherwise, the number is "read" from the string. An error gives NaN.

Boolean Conversion:

Boolean conversion is the simplest one.

It happens in logical operations but can also be performed explicitly with a call to Boolean(value).

The conversion rule:

- Values that are intuitively "empty", like 0, an empty string, null, undefined, and NaN, become false.
- Other values become true.

For instance:

```
alert( Boolean(1) ); // true
alert( Boolean(0) ); // false
alert( Boolean("hello") ); // true
alert( Boolean("") ); // false
```

Basic Operators:

Terms – unary, binary and operand:

- *An operand* – is what operators are applied to. For instance, in the multiplication of 5 * 2 there are two operands: the left operand is 5 and the right operand is 2. Sometimes, people call these "arguments" instead of "operands".

An operator is *unary* if it has a single operand.

An operator is *binary* if it has two operands.

Maths:

The following math operations are supported:

- Addition +,
- Subtraction -,
- Multiplication *,

- Division /,
- Remainder %,
- Exponentiation **.

Remainder %:

The remainder operator %, despite its appearance, is not related to percents.

The result of a % b is the remainder of the integer division of a by b.

Exponentiation **:

The exponentiation operator a ** b raises a to the power of b.

In school maths, we write that as a^b.

String concatenation with binary + :

the binary + is applied to strings, it merges (concatenates) them

Here's a more complex example:

```
alert(2 + 2 + '1' ); // "41" and not "221"
```

Here, operators work one after another. The first + sums two numbers, so it returns 4, then the next + adds the string 1 to it, so it's like 4 + '1' = '41'.

```
alert('1' + 2 + 2); // "122" and not "14"
```

Here, the first operand is a string, the compiler treats the other two operands as strings too. The 2 gets concatenated to '1', so it's like '1' + 2 = "12" and "12" + 2 = "122".

The binary + is the only operator that supports strings in such a way. Other arithmetic operators work only with numbers and always convert their operands to numbers.

Here's the demo for subtraction and division:

```
alert( 6 - '2' ); // 4, converts '2' to a number
alert( '6' / '2' ); // 3, converts both operands to numbers
```

Numeric conversion, unary + :

The unary plus or, in other words, the plus operator + applied to a single value, doesn't do anything to numbers. But if the operand is not a number, the unary plus converts it into a number.

It actually does the same thing as Number(...), but is shorter.

Operator precedence:

If an expression has more than one operator, the execution order is defined by their *precedence*, or, in other words, the default priority order of operators.

note that unary operators are higher than corresponding binary ones

Assignment = returns a value;

the result of expression (a = b + 1) is the value which was assigned to a (that is 3). It is then used for further evaluations.

```
let c = 3 - (a = b + 1);
```

Chaining assignments:

```
a = b = c = 2 + 2;
```

Chained assignments evaluate from right to left. First, the rightmost expression 2 + 2 is evaluated and then assigned to the variables on the left: c, b and a. At the end, all the variables share a single value.

Modify-in-place:

This notation can be shortened using the operators += and *=:

```
let n = 2;
n += 5; // now n = 7 (same as n = n + 5)
n *= 2; // now n = 14 (same as n = n * 2)
alert( n ); // 14
```

Such operators have the same precedence as a normal assignment, so they run after most other calculations:

```
let n = 2;
n *= 3 + 5; // right part evaluated first, same as n *= 8
alert( n ); // 16
```

Increment/decrement:

- **Increment** ++ increases a variable by 1:

```
let counter = 2;
counter++;      // works the same as counter = counter + 1, but is shorter
```

alert(counter); // 3

- **Decrement** -- decreases a variable by 1:

```
let counter = 2;
counter--;      // works the same as counter = counter - 1, but is shorter
alert( counter ); /
```

Bit-wise operators:

Bitwise operators treat arguments as 32-bit integer numbers and work on the level of their binary representation.

These operators are not JavaScript-specific. They are supported in most programming languages.

The list of operators:

- AND (&)
- OR (|)
- XOR (^)
- NOT (~)
- LEFT SHIFT (<<)
- RIGHT SHIFT (>>)
- ZERO-FILL RIGHT SHIFT (>>>)

Comma:

The comma operator , is one of the rarest and most unusual operators. Sometimes, it's used to write shorter code, so we need to know it in order to understand what's going on.

The comma operator allows us to evaluate several expressions, dividing them with a comma ,. Each of them is evaluated but only the result of the last one is returned.

```
let a = (1 + 2, 3 + 4);

alert( a ); // 7 (the result of 3 + 4)
```

Comparisons:

- Greater/less than: a > b, a < b.
- Greater/less than or equals: a >= b, a <= b.
- Equals: a == b, please note the double equality sign == means the equality test, while a single one a = b means an assignment.
- Not equals: In maths the notation is ≠, but in JavaScript it's written as a != b.

Boolean is the result:

All comparison operators return a boolean value:

- true – means "yes", "correct" or "the truth".
- false – means "no", "wrong" or "not the truth".

String Comparison:

strings are compared letter-by-letter.

For example:

```
alert( 'Z' > 'A' ); // true

alert( 'Glow' > 'Glee' ); // true

alert( 'Bee' > 'Be' ); // true
```

The algorithm to compare two strings is simple:

1. Compare the first character of both strings.

2. If the first character from the first string is greater (or less) than the other string's, then the first string is greater (or less) than the second. We're done.
3. Otherwise, if both strings' first characters are the same, compare the second characters the same way.
4. Repeat until the end of either string.
5. If both strings end at the same length, then they are equal. Otherwise, the longer string is greater.

The comparison algorithm given above is roughly equivalent to the one used in dictionaries or phone books, but it's not exactly the same.

For instance, case matters. A capital letter "A" is not equal to the lowercase "a". Which one is greater? The lowercase "a". Why? Because the lowercase character has a greater index in the internal encoding table JavaScript uses (Unicode).

- Comparison operators return a boolean value.
- Strings are compared letter-by-letter in the "dictionary" order.
- When values of different types are compared, they get converted to numbers (with the exclusion of a strict equality check).
- The values null and undefined are equal == to themselves and each other, but do not equal any other value.
- Be careful when using comparisons like > or < with variables that can occasionally be null/undefined. Checking for null/undefined separately is a good idea.

Conditional Branching:

If statement:

The if(...) statement evaluates a condition in parentheses and, if the result is true, executes a block of code.

For example:

```
let year = prompt('In which year was ECMAScript-2015 specification published?', '');

if (year == 2015) alert( 'You are right!' );
```

Boolean conversion:

The if (…) statement evaluates the expression in its parentheses and converts the result to a boolean.

the code under this condition would never execute:

```
if (0) { // 0 is falsy

  ...

}
```

…and inside this condition – it always will:

```
if (1) { // 1 is truthy
  ...
}
```

We can also pass a pre-evaluated boolean value to if, like this:

```
let cond = (year == 2015); // equality evaluates to true or false
if (cond) {
  ...
}
```

The ‘else’ clause:

The if statement may contain an optional else block. It executes when the condition is falsy.

Several conditions:

Sometimes, we’d like to test several variants of a condition. The else if clause lets us do that.

Conditional Operator ? :

Sometimes, we need to assign a variable depending on a condition.

```
let result = condition ? value1 : value2;
```

Multiple ?:

A sequence of question mark operators ? can return a value that depends on more than one condition.

```
let age = prompt('age?', 18);
let message = (age < 3) ? 'Hi, baby!' :
  (age < 18) ? 'Hello!' :
  (age < 100) ? 'Greetings!' :
  'What an unusual age!';
alert( message );
```

Logical Operators:

There are four logical operators in JavaScript: || (OR), && (AND), ! (NOT), ?? (Nullish Coalescing).

|| OR

The “OR” operator is represented with two vertical line symbols:

result = a || b;

In classical programming, the logical OR is meant to manipulate boolean values only. If any of its arguments are true, it returns true, otherwise it returns false.

a chain of OR || returns the first truthy value or the last one if no truthy value is found.

result = value1 || value2 || value3;

AND &&

The AND operator is represented with two ampersands &&

result = a && b;

The AND && operator does the following:

- Evaluates operands from left to right.
- For each operand, converts it to a boolean. If the result is false, stops and returns the original value of that operand.
- If all operands have been evaluated (i.e. all were truthy), returns the last operand.

NOT !

The boolean NOT operator is represented with an exclamation sign !.

The syntax is pretty simple:

result = !value;

The operator accepts a single argument and does the following:

1. Converts the operand to boolean type: true/false.
2. Returns the inverse value.